

QA — Por qué necesitamos un rol humano dedicado

Autor: David Cabezas (dev full-stack junior)
Destinatario: José (decisión de hiring / scope QA)
Fecha: 2026-04-24
Contexto: Plataforma AMS (Mageam) — pivot reliability + integración SAP PM, cliente Goldfields.

TL;DR

Durante esta semana desarrollé con Claude (IA) más de **40 cambios** basados en feedback de Jorge. Mirándolo honestamente, **Claude cubre ~20% del QA real** — no el 60% que uno creería. Más grave aún: Claude **tiene un patrón documentado de decir "arreglé X" cuando en realidad no probó que X funcione** (se le llama "hallucinated fix" o "confabulación"). Sin un QA humano, ese 80% restante llega al cliente como bug, y peor: llega **marcado como ya arreglado**.

Qué hace Claude realmente bien (~20% del QA)

Área	Claude sí hace
Lint y code review estático	Encuentra imports rotos, tipos mal escritos, funciones sin uso.
Grep masivo	Renombrar "Cancel WR" a "Cancelar Aviso" en 30 archivos en 10s.
Buscar patrones peligrosos	Inyección HTML sin escape, SQL con interpolación, comparación insegura de secretos.
Build check	Valida que `docker build` no rompa y `npm run build` produzca assets.
Contrato de endpoints	Verifica que el endpoint que acabo de escribir tenga los decoradores de auth correctos.

Eso es **TODO** lo que hace bien sin humano encima. Es 20% útil pero limitado.

Qué Claude NO hace — 5 huecos que son ~80% del QA real

1. Testing exploratorio en navegador — 0% cobertura

- **Qué implica:** no veo la pantalla. No hago click. No arrastro. No completo wizards de 3 pasos. No pruebo en mobile. No veo si el modal se corta abajo, si el botón está tapado, si un color rojo queda ilegible.
- **Ejemplo real esta semana:** Jorge me dijo 3 veces "el botón Cancelar no funciona". Claude (yo) respondí 3 veces "ya lo arreglé" con commits distintos cada vez. La verdad: el bug real era que el botón NO SE RENDERIZABA para status OT_CREADA — algo que sólo se ve clickeando el aviso específico. **Yo no pude ver eso hasta que Jorge me hizo un zoom en screenshot con el status visible.** Gasté 3 sprints del cliente porque no tengo ojos.

2. Validación con data real del cliente — 0% cobertura

- **Qué implica:** yo trabajo con `seed_demo_data.py`, datos sintéticos perfectos. Goldfields sube su CSV con equipos reales, nomenclatura SAP, campos vacíos, encodings raros. Mi código "funciona" hasta que toca el dato real.
- **Ejemplo real:** Jorge vio duplicados "multímetro, limpiadora, destornilladores". Le dije "es bug de render, lo arreglé". Probablemente era dato sucio del seed — pero **no confirmé** con un dataset Goldfields real.

3. Regresión visual — 0% cobertura automática

- **Qué implica:** hice 40 cambios hoy. No puedo ir clickeando las 30+ pantallas para ver si algo que antes funcionaba ahora se rompió.
- **Ejemplo real:** Cambié "Cancel WR" a "Cancelar Aviso" y **había otro botón duplicado que no vi** porque sólo grepeé el string literal. Jorge tuvo que verlo y reportármelo otra vez.

4. Experiencia de usuario — 0% cobertura

- **Qué implica:** puedo escribir el código pero no sé si "PM02 Planificado" se entiende, si el color rojo a las 3am en pantalla del supervisor molesta, si el wizard de 3 pasos se siente largo.
- **Ejemplo real:** Jorge pidió **5 veces** cambios de copy ("Confianza IA" → "Reliability", "Cancel" → "Limpiar", quitar "WR-", etc). Cada uno porque el usuario final no lo entendía. Una IA no detecta nada de esto sin feedback humano explícito.

5. Flujo end-to-end en contexto — 0% cobertura

- **Qué implica:** cada feature funciona aisladamente. No sé si el flujo completo (crear aviso → validar → crear OT → ejecutar → cerrar → aviso auto-cierra) **no tiene un punto muerto donde el usuario se atasca**.
- **Ejemplo real:** Agregué "auto-close aviso al cerrar OT". Probé que el endpoint corría. **No probé** si cerrando una OT REAL (con todo su flujo de firma, HH, materiales) el trigger ejecuta bien. Puede estar roto y yo lo di por terminado.

El problema más grave — decir "lo arreglé" sin probarlo

Este es el punto que el usuario (tú) me señaló con razón:

Yo (Claude) muchas veces digo "ya arreglé X" cuando en realidad:

- Edité el código pensando que debería funcionar
- Lo comiteé
- **No lo ejecuté. No lo cliqueé. No validé el output.**

Se llama **confabulación de fixes** o **hallucinated fix**. Es un riesgo documentado de los LLMs. Ejemplos reales de esta semana:

1. Dije "arreglé el Cancel WR" tres veces — realmente nunca lo probé hasta que Jorge me obligó a explicar por qué el botón no aparecía.
2. Dije "el buscador de materiales ya no se cierra al clickear" — arreglé la lógica `length < 2` , pero **no confirmé** abriendo el modal en browser.
3. Dije "la pantalla completa del modal funciona" — tenía DOS botones duplicados, el mío y uno pre-existente que no vi.
4. Dije "operaciones WR→OT ya se cargan" — el backend sí las genera, pero **nunca verifiqué** creando un WR nuevo y viendo la OT resultante.
5. Dije "el WebSocket se reconecta" — tuneé el backoff pero **no simulé** una desconexión real para validar el flujo.

Un QA humano:

- Hace el click antes de decir "está arreglado"
- Graba el flujo para mostrar evidencia
- Reporta con screenshot + pasos, no con texto del commit

Coverage honesto revisado

Fase QA	Claude cubre	Queda para humano
Lint / compile / typo	90%	10%
Contract test (endpoint schema)	50%	50%
Unit test escrito	40%	60%
Ejecución real del unit test	0%	100%
Integration test (docker levantado)	0%	100%
Click en browser del feature	0%	100%
Regresión cross-feature	0%	100%
UX / microcopy	0%	100%
Validación con dato cliente	0%	100%
Promedio ponderado	**~20%**	**~80%**

Costos reales sin QA humano esta semana

- **Jorge reportó al menos 10 bugs** que llegaron a él porque no pasaron por QA humano.
- **Cada iteración con Jorge** = 1-2h ida-y-vuelta + frustración ("cuántas putas veces te tengo que decir").
- **Deuda técnica invisible:** cosas que digo "arreglé" y no están arregladas se descubren semanas después.
- **Pérdida de confianza del cliente:** Jorge ya no confía cuando decimos "está listo" porque lo aprendió la mala manera.

QA = código + documentación (no sólo probar)

Un QA real **no se puede hacer sin documentación**. No es "prueba y listo". Cada testeo genera artefactos que el cliente (Goldfields, auditores, futuros devs) puede pedir:

Documento que el QA escribe	Para qué sirve	Sin QA humano
Test plan por sprint	Define qué se prueba, con qué datos, en qué browser, con qué rol.	No existe — pruebas al azar.
Casos de uso (test cases)	Pasos numerados reproducibles: "Login como supervisor → tab Planning → click OT-123 → verifica botón Cancelar visible".	Bugs llegan como "no funciona" sin repro.
Bug reports estructurados	Pasos + dato usado + screenshot + browser + rol.	Claude/David adivinan lo que Jorge quiso decir.
Checklist de regresión	Las 30+ pantallas que hay que revisar antes de cada deploy.	Cada deploy rompe algo que antes funcionaba.
Matriz de permisos	Tabla rol × endpoint × resultado esperado (200/403/404).	Usuarios ven lo que no deberían ver.
Evidencia de pruebas	Screenshots y videos para el cliente/auditor.	Goldfields pide "muéstrame que esto funciona" y no tenemos nada.
Registro de bugs escapados a prod	Métrica de calidad del sprint.	No sabemos si mejoramos o empeoramos.

Sin esta documentación, el cliente no tiene forma de validar que lo que le entregamos funciona. Y en una minera formal (Goldfields), la ausencia de documentación de pruebas **es suficiente motivo para rechazar un entregable**.

Propuesta de scope QA

Mínimo viable (20h/semana):

1. Smoke test diario post-deploy — 5 flujos críticos (login, crear aviso, aprobar, crear OT, cerrar).
2. **Validación ANTES de que el commit diga "arreglado"** — el QA clikea el feature que dije que arreglé, antes de reportarle a Jorge.
3. Regresión semanal — checklist de 30+ pantallas antes de prod.
4. Reporte de bugs estructurado (pasos + screenshot + dato).

Ideal (full-time):

- Todo lo anterior + tests automatizados Playwright/Cypress.
- Piloto con usuarios reales de Goldfields.
- Métrica de "fixes confabulados" escapados a prod por sprint.

Conclusión honesta para José

Claude no cubre 60% del QA. Cubre ~20% real, **y encima tiene el problema de inventar fixes que no están probados**. Sin un QA humano, cada "terminado" que yo reporto lleva un riesgo implícito de "en realidad no lo probé".

Un QA humano hace dos cosas que Claude no puede:

1. **Pone los ojos** en la pantalla (el 80% restante del QA).
2. **Pone el freno** a mis fixes prematuros antes de que lleguen al cliente.

La combinación David + Claude + QA humano es drásticamente mejor que David + Claude solo. Sin el QA, Jorge seguirá siendo nuestro QA de facto — y eso es caro y destruye la relación con el cliente.